# Continuous Median

AlgoExpert · Hard · Heaps

# The problem

- Insert numbers one at a time and query the running median, using a max-heap for the lower half and a min-heap for the upper half.

# Intuition

- Keep the smaller half in a max-heap and the larger half in a min-heap, balanced in size.

# Brute force vs optimal

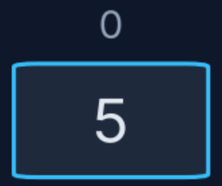
- Keep a sorted list, insert in place → O(n) per insert
- Two heaps (max-heap low, min-heap high) → O(log n) insert, O(1) median
- Balance sizes within 1

# Approach

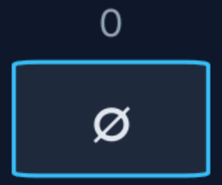
- 1. Insert into the right heap
- 2. Rebalance so sizes differ by at most one
- 3. Median is the top of the bigger heap or the average of tops

# Worked example

lower half (max-heap)




upper half (min-heap)

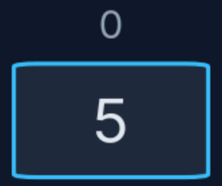



```
insert 5 → median 5
```

Insert 5: median 5

# Worked example

lower half (max-heap)




upper half (min-heap)

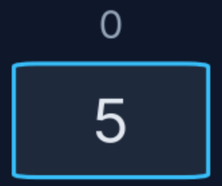



```
insert 10 → median 7.5
```

Insert 10: median 7.5

# Worked example

lower half (max-heap)




upper half (min-heap)




```
insert 100 → median 10
```

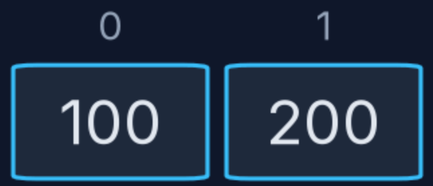
Insert 100: median 10

# Worked example

lower half (max-heap)




upper half (min-heap)




```
insert 6 → median 10
```

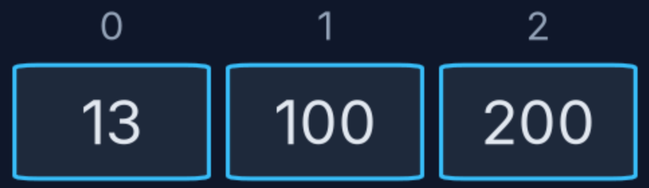
Insert 6: median 10

# Worked example

lower half (max-heap)




upper half (min-heap)




```
insert 13 → median 11.5
```

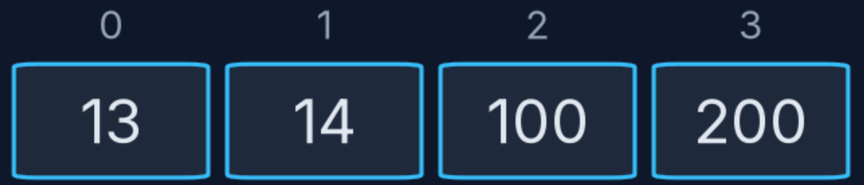
Insert 13: median 11.5

# Worked example

lower half (max-heap)




upper half (min-heap)




```
insert 14 → median 13
```

Insert 14: median 13

# Java solution

```java
public class ContinuousMedian {
    public static class ContinuousMedianHandler {
        private final PriorityQueue<Integer> lower = new PriorityQueue<>(Collections.reverseOrder());
        private final PriorityQueue<Integer> upper = new PriorityQueue<>();
        private double median;

        public void insert(int n) {
            if (lower.isEmpty() || n < lower.peek()) lower.add(n); else upper.add(n);
            if (lower.size() > upper.size() + 1) upper.add(lower.poll());
            else if (upper.size() > lower.size() + 1) lower.add(upper.poll());
            if (lower.size() == upper.size()) median = (lower.peek() + (double) upper.peek()) / 2;
            else median = lower.size() > upper.size() ? lower.peek() : upper.peek();
        }

        public double getMedian() { return median; }
    }
}
```

# Complexity

- Time: O(log n) insert, O(1) median
- Space: O(n)

# Edge cases & pitfalls

- Rebalance after every insert
- Even count → average
- Use double division

# Interview tips & follow-ups

- Follow-up: sliding-window median

# Recap

- Keep the smaller half in a max-heap and the larger half in a min-heap, balanced in size.
- Complexity: O(log n) insert, O(1) median